01-guides-and-plan

01 — Guides & Plan extraction

> Exhaustive extraction of the BobaLink reference deliverable's top-level guide, the 5 formal docs, README, and the two plan documents. Every functional/non-functional requirement, feature, flow, edge case, config, API endpoint, dependency, build/release step, acceptance criterion, milestone, and backend-integration point is captured verbatim with file + section citation. Boba-specific constraint flags are called out explicitly. Source product is named **BobaLink**, v1.0.0, dated 2026-06-06. NOTE: BobaLink is a *reference/spec deliverable*, not the Boba repo's own naming — the Boba implementation will rename/re-target.

Files read

#	File	Lines
1	`Browser Torrent Extension Guide.md` (master guide / index)	240
2	`Browser Torrent Extension Guide/docs/technical-specification.md`	1152
3	`Browser Torrent Extension Guide/docs/api-reference.md`	2042
4	`Browser Torrent Extension Guide/docs/developer-guide.md`	1334
5		949

#	File	Lines
	`Browser Torrent Extension Guide/docs/installation-guide.md`	
6	`Browser Torrent Extension Guide/docs/user-guide.md`	865
7	`Browser Torrent Extension Guide/README.md` (byte-identical to file #1)	240
8	`Browser Torrent Extension Guide/plan.md` (master execution plan)	96
9	`Browser Torrent Extension Guide/implementation-plan.md`	410
	TOTAL	**7328**

All 9 files read IN FULL (chunked to EOF). Files #1 and #7 are byte-for-byte identical (the master guide is also the README).

A. PROJECT OVERVIEW & KEY METRICS

Source: master-guide §“Project Overview”; tech-spec §1 “Executive Summary”; README.

- BobaLink = cross-browser WebExtension (**Manifest V3**) that detects magnet links and `.torrent` file references on any web page (and across tab groups), deduplicates by cryptographic infohash, and transmits them to a **Boba orchestration server** OR directly to a **qBitTorrent WebUI** instance.
- Three core operational problems it solves (tech-spec §1): (1) Discovery-to-Download latency (eliminate manual copy/paste — reduce to single click); (2) Batch operation efficiency (tab groups as atomic units); (3) Operational resilience (offline-aware queue with exponential backoff so transient failures never lose torrents).
- Does NOT perform any BitTorrent protocol operations itself — it is a “thin client” / “intelligent scraper and forwarding agent” (tech-spec §2).

- Built on **WXT framework** + **TypeScript**; targets Chrome (≥88), Firefox (≥109), Opera, Yandex Browser; WCAG 2.1 AA; credentials encrypted at rest via **AES-256-GCM** (Web Crypto API).

Key Metrics at a Glance (tech-spec §1 table): | Metric | Target | |—|—| | Extension bundle size (compressed) | ≤ 350 KB | | Cold-start service worker | ≤ 50 ms | | Magnet link detection latency | ≤ 5 ms per link | | API call timeout | 30 s | | Offline queue retention | 30 days | | Supported browsers | 4 (Chrome, Firefox, Opera, Yandex) | | Functional requirements | 25 | | Non-functional requirements | 15 |

Deliverable inventory (master-guide §"File Count Summary"/"Key Statistics"): 60 source files; 36 diagrams (12 types × 5 formats); 5 markdown docs (25,725+ words); research = 27,560 lines across 12 dimensions; 320+ test cases (unit+E2E); 80%+ coverage target; ~4.8 MB total. **133 files / 8 formats.**

B. FUNCTIONAL REQUIREMENTS (25 FRs — tech-spec §3, verbatim)

B.1 Torrent Detection & Parsing (§3.1)

- **FR-001 — Detect Magnet Links on Any Web Page** (P0 Critical; Content Script; trigger = DOM mutation via `MutationObserver` or page load). SHALL detect all magnet URI links (`href` beginning `magnet:?`) in DOM of any page; detection MUST be dynamic across SPA navigation. **Acceptance:** magnet links in dynamically injected content (React/Vue re-renders) detected within **500 ms**; works across all `http:`/`https:` origins; iframe magnet links detected when `all_frames` enabled.
- **FR-002 — Detect .torrent File Links** (P0 Critical; Content Script). Detect via URL path suffix `.torrent` (case-insensitive) AND `Content-Type: application/x-bittorrent` response header (when accessible via CORS or Fetch API).
- **FR-003 — Parse Magnet URI Parameters** (P0 Critical; Parser Module). Per **BEP-0009**: `xt` (Exact Topic, `urn:btih:` + 40-char hex infohash), `dn` (Display Name), `tr` (Tracker URL, repeatable), `xl` (Exact Length bytes, optional), `ws` (Web Seed, optional), `x.pe` (Peer Address host:port, optional). **Validation:** `xt` MUST be present + valid 40-char hex infohash; `dn` SHOULD be present, fallback `"Unknown Torrent"`; `tr` entries MUST be valid HTTP(S) URLs, malformed trackers logged+discarded.

- **FR-004 — Download and Parse .torrent Files** (P1 High; Service Worker). Download via `fetch()`. Constraints: **max file size 10 MB** (configurable via `maxTorrentFileSize`); **timeout 30 s**; CORS policy falls back to **proxying via Boba server** when direct fetch blocked. Parse via Bencode decoder to extract `info.name`, `info.piece length`, `info.length`(or `info.files` multi-file), `announce` + `announce-list`.
- **FR-005 — Compute Infohash from Torrent Data** (P1 High; Crypto Module). Compute **SHA-1** over Bencode-encoded `info` dict → 40-char hex infohash; MUST match `xt` if both available.
- **FR-006 — Deduplicate Torrents by Infohash** (P1 High; State Manager). In-memory `Map<string, TorrentInfo>` keyed by **lowercase infohash**. On duplicate (same infohash, different page): preserve most descriptive `displayName` (longest string), union of all tracker URLs, earliest `detectedAt`.

B.2 Integration and Transmission (§3.2)

- **FR-007 — Send Magnet Links to qBitTorrent via WebUI API** (P0 Critical; API Client). Transmit to qBitTorrent `/api/v2/torrents/add`; support `urls` param (newline-separated magnets), optional `category`/`tags`/`savepath`/`rename`, automatic cookie-based session management.
- **FR-008 — Upload .torrent Files to qBitTorrent** (P1 High; API Client). Upload via `multipart/form-data` POST to `/api/v2/torrents/add` using `torrents` file field. File blobs kept in memory only for upload duration.
- **FR-009 — Support Tab Group Batch Operations** (P1 High; Tab Manager). Enumerate all tabs in a selected Chrome tab group; send `SCAN_REQUEST` to each tab's content script; aggregate `SCAN_RESPONSE`; dedup across tabs; transmit unified list as batch.
- **FR-010 — Enumerate Tab Groups and Extract URLs** (P1 High; Tab Manager). Using `chrome.tabGroups.query()` + `chrome.tabs.query()`: list all tab groups in current window; display group titles + colors in popup; allow selecting one or more groups for batch.

B.3 Service Discovery and Authentication (§3.3)

- **FR-011 — Auto-Discover Boba Services on Local Network** (P2 Medium; Discovery Module). When no server configured: probe `https://boba.local:8443/health`, `https://boba.local:8080/health`, `https://boba:8443/health`; `fetch()` with `mode:'no-cors'` + 5 s timeout; parse version, validate min supported `>=1.0.0`; present ranked list (fastest first); persist selection to encrypted storage.

(NOTE — Boba flag: real Boba ports are 7186/7187/7189, NOT 8443/8080. See §J.)

- **FR-012 — Support Multiple Authentication Methods** (P1 High; Auth Module). Methods: **Cookie-based** (auto `SID` cookie from qBitTorrent login — default for qBT direct); **API Key** (`X-API-Key` header for Boba server — default for Boba mode); **Basic Auth** (`Authorization: Basic <base64>` — optional); **Custom Header** (user-defined name+value — optional).
- **FR-013 — Encrypt Stored Credentials** (P0 Critical; Crypto Module). All sensitive data to `chrome.storage.local` encrypted **AES-256-GCM** (Web Crypto API): 256-bit master key derived from per-installation random salt via **PBKDF2**; encryption key stored in `chrome.storage.session` (in-memory only, cleared on browser restart); each credential gets a unique **96-bit IV**; **128-bit auth tag** prepended to ciphertext.

B.4 Offline and Resilience (§3.4)

- **FR-014 — Offline Queue with Retry** (P1 High; Queue Manager). Persistent FIFO queue for failed sends: **max queue size 1,000** (configurable); retry = **exponential backoff starting 5 s, capped at 5 min**; **max retry attempts 5** (configurable); persisted to `chrome.storage.local`, survives restarts; items past max retries → **"Dead Letter"** sub-store for manual review.
- **FR-015 — Real-time Download Progress via Badge** (P2 Medium; Badge Manager). Toolbar badge displays numeric count of queued torrents; colors: `green`(idle), `blue`(sending), `orange`(queued/retrying), `red`(error); updates within **200 ms** of state change.

B.5 User Interface (§3.5)

- **FR-016 — Context Menu Integration** (P1 High; UI Controller). Via `chrome.contextMenus`: Link(magnet)→"Send Magnet to Boba"; Link(.torrent)→"Download .torrent to Boba"; Page→"Scan Page for Torrents"; Tab Group→"Send Tab Group to Boba".
- **FR-017 — Keyboard Shortcuts** (P2 Medium; Command Handler). Via `chrome.commands`: `send-current-page` = `Ctrl+Shift+B`(`Cmd+Shift+B` mac); `open-popup` = `Ctrl+Shift+L`(`Cmd+Shift+L`); `send-tab-group` = `Ctrl+Shift+G`(`Cmd+Shift+G`). All user-customizable via `chrome://extensions/shortcuts`. (NOTE: user-guide §2.7 adds a 4th: "Scan page (without sending)" = `Ctrl+Shift+S`/`Cmd+Shift+S` — see §D.13 ambiguity.)
- **FR-018 — Cross-Browser Compatibility** (P0 Critical; Build System). Single codebase targets: Chrome 88+ (Blink, primary dev target);

Firefox 109+ (Gecko, polyfill for `chrome.*`); Opera 74+ (Blink, Chromium-compat); Yandex 21+ (Blink, Chromium-compat). Browser-specific code isolated behind `BrowserAdapter` abstraction.

- **FR-019 — Internationalization Support** (P2 Medium; I18n Module). Via `_locales/{lang}/messages.json`: `en`(default), `zh_CN`, `zh_TW`, `es`, `fr`, `de`, `ja`, `ko`. All user-facing strings externalized; no hardcoded UI text.
- **FR-020 — Accessibility Compliance (WCAG 2.1 AA)** (P1 High; UI Layer). Contrast ≥4.5:1 normal text / 3:1 large; all interactive elements keyboard-navigable (Tab order, Enter/Space); ARIA labels on icon-only buttons; screen-reader announcements via `aria-live`; visible focus indicators.
- **FR-021 — Dark/Light Theme** (P2 Medium; UI Layer). Respect `prefers-color-scheme` + manual override toggle in Options; CSS custom properties for all theme-aware colors.
- **FR-022 — Notifications for Download Events** (P2 Medium; Notification Manager). Via `chrome.notifications`: Send success→"Torrent Sent"; Send failed→"Send Failed" (click to retry); Batch complete→"Batch Complete" (`{count}` sent, `{failed}` failed); Queue retry→"Retrying Send" (attempt `{n}`). Suppressible per-category in Options.
- **FR-023 — Options Page Configuration** (P1 High; Options Page). Provides: Server URL + connection mode (Boba proxy / qBitTorrent direct); auth credentials (masked input + test-connection button); default category/tags/download path; notification prefs; queue settings (max size, retry count, backoff strategy); keyboard shortcut display (non-editable, links to browser shortcuts page); theme (system/light/dark); reset to defaults with confirmation.
- **FR-024 — Health Check and Connection Status** (P1 High; Health Monitor). Persistent status indicator: poll interval **30 s** (configurable); endpoint `/api/v2/app/version` (qBitTorrent) or `/health` (Boba); states `connected`/`connecting`/`disconnected`/`error`; visual indicator in popup header; tooltip shows last successful ping time.
- **FR-025 — Rate Limiting for API Calls** (P1 High; API Client). Client-side: default **10 req/s burst, 60 req/min sustained**; queue processing = max 1 concurrent API call + 500 ms inter-call delay; respects HTTP 429 `Retry-After`; rate-limit state in-memory (non-persistent).

C. NON-FUNCTIONAL REQUIREMENTS (15 NFRs — tech-spec §4)

C.1 Performance (§4.1)

ID	Requirement	Target	Measurement
NFR-001	Content script initialization	≤ 10 ms	`performance.now()` delta
NFR-002	Magnet link detection (per link)	≤ 5 ms	benchmark on 1,000-link page
NFR-003	Popup render time	≤ 100 ms	Lighthouse perf audit
NFR-004	Options page load time	≤ 200 ms	Lighthouse perf audit
NFR-005	API call round-trip (local network)	≤ 500 ms	p95 over 100 calls
NFR-006	Service worker cold start	≤ 50 ms	Chrome DevTools Performance
NFR-007	Bundle size (compressed CRX)	≤ 350 KB	`du -h` on artifact

C.2 Security (§4.2)

ID	Requirement	Implementation
NFR-008	Credential encryption at rest	AES-256-GCM via Web Crypto API
NFR-009	No cleartext credential logging	ESLint rule + code review
NFR-010	HTTPS-only API communication	URL scheme validation + CSP
NFR-011	Content Security Policy	`script-src 'self'; object-src 'none'`
NFR-012	Minimum permission model	`activeTab` + declared host permissions only

C.3 Reliability (§4.3)

ID	Requirement	Target
NFR-013	Crash-free session rate	≥ 99.9%
NFR-014	Offline queue durability	100% — all items survive restart
NFR-015	API compatibility coverage	qBitTorrent 4.4.x through 5.x

C.4 Scalability (§4.4)

- Concurrent tab group batch size: **unlimited** (memory-bound).
- Offline queue: **1,000 default, 10,000 maximum**.
- Detection: pages with 10,000+ links must not cause UI jank (> 16 ms frame time).

C.5 Maintainability (§4.5)

- Coverage: **≥ 80% unit, ≥ 60% E2E scenario**.
- All public APIs documented w/ JSDoc.
- TypeScript strict mode; **zero `any` in production**.
- **≤ 20 runtime dependencies**.

D. DATA MODELS & TYPE DEFINITIONS

D.1 TorrentInfo (tech-spec §6.1; api-ref §5.2)

`infohash` (40-char lowercase hex), `displayName`, `magnetUri`, `source` (`'magnet-link' | 'torrent-file'`), `trackers: string[]`, `totalSize?: number`, `webSeeds?: string[]`, `pageUrl`, `detectedAt` (Unix ms), `fileBlob?: Blob` (.torrent uploads only).

D.2 ServerConfig (§6.2; api-ref §5.2)

`mode` (`'boba' | 'qbittorrent-direct'`), `baseUrl`, `authMethod` (`'cookie'|'api-key'|'basic'|'custom-header'`), `credentials: EncryptedCredentials`, `defaultCategory?`, `defaultTags?: string[]`, `defaultSavePath?`, `timeout`, `healthCheckInterval`.

D.3 ExtensionConfig (§6.3; api-ref §5.2)

`version`, `server: ServerConfig`, `ui` {`theme: 'system'|'light'|'dark'`, `badgeEnabled`, `notifications`{`sendSuccess`,`sendFailed`,`batchComplete`,`queueRetry`}}, `queue`{`maxSize`,`maxRetries`,`retryBaseDelayMs`,`retryMaxDelayMs`}, `detection`{`scanDynamically`,`maxTorrentFileSize`,`enableTabGroupScan`}, `rateLimit`{`requestsPerSecond`,`requestsPerMinute`,`interCallDelayMs`}.

D.4 QueueItem (§6.4; api-ref §5.3)

`id`, `torrent: TorrentInfo`, `status` (`'pending'|'retrying'|'failed'|'dead-letter'`), `attemptCount`, `nextRetryAt` (Unix ms), `lastError?`, `lastHttpStatus?`, `queuedAt`.

D.5 EncryptedCredentials (api-ref §5.2)

`salt` (base64, 16 bytes), `ciphertext` (base64 = IV 12 bytes + ciphertext + authTag 16 bytes), `encryptedAt`.

D.6 Other types (api-ref §5)

Scalar types: `Infohash` (`/^[a-f0-9]{40}$/`), `MagnetUri` (`/^magnet:\?xt=urn:btih:[a-fA-F0-9]{40}/`), `Priority` (`'low'|'normal'|'high'|'critical'`), `ConnectionState`, `AuthMethod`, `ConnectionMode`, `ThemePreference`, `QueueStatus`, `TorrentSource`. Also `ApiResponse<T>`, `ApiError`, `ConnectionTestResult`, `QueueSnapshot`, `RateLimitBucket`, `DeepPartial<T>`, `MessageHandler<>`. Full qBitTorrent types: `QBitTorrentInfo` (40+ fields), `TorrentState` (18-value union: error, missingFiles, uploading, pausedUP, queuedUP, stalledUP, checkingUP, forcedUP, allocating, downloading, metaDL, pausedDL, queuedDL, stalledDL, checkingDL, forcedDL, checkingResumeData, moving), `ServerState`, `MainDataResponse`.

D.7 SQL Schema (implementation-plan §“SQL Schema Definitions”)

Models `chrome.storage.local` as SQL-shaped tables:
`extension_config(key PK, value, updated_at)`;
`discovered_torrents(id=infoHash PK, page_url, magnet_uri, torrent_url, name, trackers JSON, size_bytes, source_type CHECK('magnet-link'|'torrent-file'|'torrent-url'), discovered_at, sent_to_boba, boba_status`

CHECK('pending'|'queued'|'added'|'error')); download_queue(id AUTOINC, info_hash, magnet_uri, torrent_data BLOB base64, name, category, save_path, added_at, retry_count, last_error, status CHECK('pending'|'retrying'|'failed'|'completed')); server_config(id=1, base_url, auth_method CHECK('none'|'cookie'|'api_key'|'basic'), api_key, username, password_encrypted, is_reachable, last_check, qbittorrent_version, boba_version); send_history(id AUTOINC, info_hash, name, sent_at, success, error_message, server_url); optional search_cache(id AUTOINC, query, category, results_json, cached_at, expires_at). **NOTE:** schema vocabularies disagree with TS types (source_type adds 'torrent-url'; status uses 'completed' not 'dead-letter'; auth_method adds 'none', drops 'custom-header') — see §K ambiguities.

E. APIs / ENDPOINTS

E.1 Internal message-passing protocol (tech-spec §7.1; api-ref §2)

Uniform envelope ExtensionMessage<T> { type: MessageType, requestId?, payload: T, sender?{tabId?,frameId?,url?} }. **MessageType union (23 values):** SCAN_REQUEST, SCAN_RESPONSE, TORRENT_DETECTED, SEND_TORRENT, SEND_BATCH, GET_STATUS, STATUS_RESPONSE, GET_QUEUE, QUEUE_RESPONSE, RETRY_ITEM, REMOVE_ITEM, CLEAR_QUEUE, GET_CONFIG, CONFIG_RESPONSE, UPDATE_CONFIG, CONFIG_UPDATED, GET_HEALTH, HEALTH_RESPONSE, DISCOVER_SERVERS, DISCOVERY_RESPONSE, TEST_CONNECTION, CONNECTION_TEST_RESULT.

Per-message payloads (api-ref §2.2–2.10): - SCAN_REQUEST (SW→CS via chrome.tabs.sendMessage): {requestId, deepScan, scope?}. - SCAN_RESPONSE (CS→SW): {requestId, torrents: TorrentInfo[], error?, meta:{linksScanned, magnetsFound, torrentFilesFound, scanDurationMs}}. - TORRENT_DETECTED (CS→SW via chrome.runtime.sendMessage): {torrents, pageUrl, timestamp}. - SEND_TORRENT (Popup/CtxMenu→SW): {torrent, priority?: 'normal'|'high', category?, tags?}. - SEND_BATCH (TabMgr→SW): {torrents, category?, tabGroupId?}; response {accepted, rejected, failed, results:[{infohash, status:'queued'|'sent'|'duplicate'|'failed', error?}]}. - GET_STATUS→STATUS_RESPONSE: {connection, serverUrl, serverVersion?, queueSize, pendingCount, deadLetterCount, lastHealthCheck?, extensionVersion}. - GET_QUEUE: {filter?, limit?

(def 100), offset?} → {items, total, filtered}. - RETRY_ITEM {itemId}; REMOVE_ITEM {itemId}; CLEAR_QUEUE {status?}. - GET_CONFIG (empty)→full ExtensionConfig; UPDATE_CONFIG {config: DeepPartial<ExtensionConfig>} (SW validates, merges, persists encrypted, broadcasts CONFIG_UPDATED). - DISCOVER_SERVERS {timeout? (def 5000), targets?} → DISCOVERY_RESPONSE {servers: [{url,name,version,responseTimeMs,healthy}], targetsProbed, durationMs}.

E.2 Boba API integration (tech-spec §7.2; api-ref §3) — base `https://{host}:{port}/api/v1`

- POST /api/v1/auth/token — body {apiKey} → {token(JWT), expiresAt, type:"Bearer"}; 401 → {error:"E_AUTH_INVALID_KEY", message}.
- POST /api/v1/auth/refresh — Authorization: Bearer {jwt} → {token, expiresAt}.
- GET /api/v1/health — no auth → {status:"healthy", version, uptime, extensions:{qbittorrent, search, metadata}}.
- GET /api/v1/torrents/search?q=&category=&limit=(def 50,max 100)&offset= — header X-API-Key → {results:[{infohash, displayName, magnetUri, size, seeders, leechers, source, category, uploadedAt}], total, query, categories}.
- POST /api/v1/torrents/download — X-API-Key + JSON {magnetUri?|infohash?, category?, tags?, savePath?, rename?, paused?, skipChecking?, sequentialDownload?, firstLastPiecePrio?} → 202 {jobId, status:"queued", infohash, estimatedStart}; 409 → {error:"E_DUPLICATE", message, existingInfohash}.
- GET /api/v1/events/download-progress — X-API-Key, Accept: text/event-stream (SSE). Event types: progress{infohash, progress(0–1), speed, eta, downloaded, total, numSeeds, numLeechs, state}, completed{infohash, completionTime, ratio}, error{infohash, error, code}, stopped{infohash, reason}.

E.3 qBitTorrent WebUI API v2 (api-ref §4) — base e.g. `https://localhost:8080`

ALL endpoints require `Referer` header matching qBitTorrent origin (else rejected); session via SID cookie. - POST /api/v2/auth/login — form username,password; 200 body Ok.SID; 403 invalid; 429 too many attempts. - POST /api/v2/auth/logout. - POST /api/v2/torrents/add — params: urls(newline-sep magnets/HTTP*) OR torrents(multipart file*) [≥1 required], savepath, cookie, category, tags(comma-sep),

skip_checking, paused, root_folder, rename, upLimit, dlLimit, ratioLimit, seedingTimeLimit, autoTMM, sequentialDownload, firstLastPiecePrio. Responses: 200 ok, 400 missing urls/torrents, 403 not auth, 415 unsupported media type, 500. - GET /api/v2/torrents/info?filter=&category=&tag=&sort=&reverse=&limit=&offset=&hashes= (filter values: all, downloading, seeding, completed, paused, active, inactive, resumed, stalled, stalled_uploading, stalled_downloading, errored; hashes pipe-separated). - POST /api/v2/torrents/delete — hashes(pipe-sep or all), deleteFiles. - POST /api/v2/torrents/pause — hashes. - POST /api/v2/torrents/start — hashes. - POST /api/v2/torrents/recheck — hashes. - POST /api/v2/torrents/reannounce — hashes. - GET /api/v2/torrents/categories. - POST /api/v2/torrents/createCategory — category, savePath?. - GET /api/v2/torrents/tags. - POST /api/v2/torrents/addTags — hashes, tags. - POST /api/v2/torrents/removeTags (listed in §4.7 summary). - GET /api/v2/sync/maindata?rid= (0 = initial, then last rid). - GET /api/v2/app/version (plain text e.g. v4.6.4). - GET /api/v2/app/webapiVersion (plain text e.g. 2.9.3). - GET /api/v2/app/preferences | POST /api/v2/app/setPreferences (json=). - GET /api/v2/app/buildInfo (qt/libtorrent/boost/openssl/zlib). - GET /api/v2/transfer/info.

E.4 Base URLs (api-ref §“Base URLs”)

Env	Boba	qBitTorrent
Local (default)	https://boba.local:8443	https://localhost:8080
Docker	https://boba:8443	https://qbittorrent:8080
Custom	user-configured	user-configured

F. ERROR HANDLING

F.1 Tech-spec §9.1 categories

E_NETWORK (backoff, silent/badge), E_TIMEOUT (immediate retry ×2 then backoff, silent), E_AUTH (no retry, immediate notification), E_RATE_LIMIT/429 (honor Retry-After then backoff, silent), E_VALIDATION (no retry, immediate notification), E_SERVER/5xx (backoff+jitter, badge+optional notif), E_STORAGE (no retry, immediate notification). ExtensionError{code, message, timestamp, context?, recoverable}.

F.2 api-ref §6.1 full error-code catalogue (22 codes)

`E_UNKNOWN`, `E_NETWORK`, `E_TIMEOUT`(408), `E_DNS`, `E_CONN_REFUSED`, `E_AUTH`(401), `E_AUTH_EXPIRED`(401), `E_RATE_LIMIT`(429), `E_CLIENT_RATE_LIMIT`, `E_VALIDATION`(400), `E_SERVER`(500), `E_SERVICE_UNAVAILABLE`(503), `E_TORRENT_INVALID`(400), `E_TORRENT_DUPLICATE`(409), `E_STORAGE_FULL`, `E_STORAGE_CORRUPT`, `E_CRYPTO`, `E_PERMISSION_DENIED`(403), `E_TAB_ACCESS`, `E_CORS`, `E_FILE_TOO_LARGE`(413), `E_UNSUPPORTED_BROWSER`. Boba error body: `{error, message, retryAfter?, details{limit, window, remaining}}`.

F.3 Retry strategies (api-ref §6.4)

Network: backoff, 5 retries (5→10→20→40→80 s). Timeout: backoff, 3 (2→4→8 s). Rate-limit/429: honor Retry-After, 10. Auth: no retry. 5xx: backoff+jitter, 5 (5→10→20→40→80 s). Validation: no retry.

G. CONFIG / SETTINGS (Options page — user-guide §3.1, installation-guide §5)

Server Settings: Connection Mode (Boba/qBT-direct), Server URL, Authentication (API Key / Username+Password / Basic / Custom Header), Test Connection, Connection Timeout (default **30 s**), Health Check Interval (default **30 s**). **Download Preferences:** Default Category (—), Default Tags (—; e.g. `browser,auto`), Default Save Path (—; server-side path), Pause After Add (Off), Skip Hash Check (Off), Sequential Download (Off), First/Last Piece Priority (Off). **Queue Settings:** Max Queue Size (**1,000**), Max Retries (**5**), Base Retry Delay (**5 s**), Max Retry Delay (**300 s**). **Notification Settings:** Send Success (Enabled), Send Failed (Enabled), Batch Complete (Enabled), Queue Retry (Disabled). **Detection Settings:** Dynamic Scanning (On), Highlight on Page (Off), Max File Size MB (**10**), Tab Group Scanning (On). **UI Settings:** Theme (System/Light/Dark; default System), Show Badge (On), Badge Color (Default; per-state custom). **Security Settings:** Require Password on Startup (Off), Auto-Lock after N min inactivity (**30**), HTTPS Only (On), Certificate Pinning (Off). “Reset to Defaults” with confirmation; “Forgot Password” → no recovery, must reset extension.

H. USER-FACING FLOWS & USE CASES

H.1 Core workflows (tech-spec §2.2)

- **W1 Single magnet detect+forward:** content script observes DOM mutations → match regex `^magnet:\?xt=urn:btih:[a-fA-F0-9]{40}` → parse to `TorrentInfo` → `chrome.runtime.sendMessage()` → SW enqueues or sends per connectivity → target API initiates download.
- **W2 Tab group batch ingestion:** user triggers “Send Tab Group to Boba” → SW enumerates group tabs → each gets `SCAN_REQUEST` → aggregate, dedup by infohash, transmit batch.
- **W3 Auto-discovery:** on first install / empty config, send mDNS/ Bonjour-like probes to well-known local addresses → responsive Boba instances reply endpoint+version → user picks from ranked list → persists to encrypted storage.

H.2 Send methods (user-guide §2.2)

1. Context menu right-click “Send Magnet to Boba”; (2) Popup → check torrents → “Send Selected”; (3) keyboard `Ctrl+Shift+B`; (4) Popup “Send All”.

H.3 .torrent file send (user-guide §2.3)

Context menu “Download .torrent to Boba” downloads (≤10 MB), parses, sends. CORS fallback: Boba mode auto-proxies; qBT-direct shows “CORS blocked” → user copies link to qBT manually.

H.4 Tab groups batch (user-guide §2.4)

Methods: Popup “Tab Groups” tab → “Send” next to group; right-click tab “Send Tab Group to Boba”; `Ctrl+Shift+G`. Result: scans every tab, collects, removes cross-tab dups, batch sends, summary notif (“Batch Complete — 12 sent, 0 failed”).

H.5 Queue management (user-guide §2.5)

Popup “Queue” tab — columns Name/Status/Attempts/Next Retry/Actions. Actions: Retry now (circular arrow), Remove (X, no recovery), Retry All Failed, Clear Queue (confirm). Behavior: auto-retry exponential backoff (5/10/20/40/80 s); max 5 → Dead Letter; dead-letter requires manual action; queue persists across restarts/reboots.

H.6 Download progress (user-guide §2.6)

Toolbar badge (number = queued/sending count; colors green/blue/orange/red/gray). Popup “Downloads” tab (SSE, Boba mode only): Name/Progress bar+%/Speed/ETA/Status. qBT-direct = no live progress in popup (check qBT WebUI).

H.7 First-time setup wizard (user-guide §1.4; installation-guide §4)

Step 1 choose Connection Mode (Boba Server recommended / qBitTorrent Direct); Step 2 (Boba) server discovery → “Discover Servers” scans `boba.local:8443`,`boba:8443`, ranked list, “Connect”; manual fallback (URL + API key + Test Connection); Step 3 (qBT-direct) URL + username/password + Test Connection + optional default category/path; Step 4 “Finish Setup”.

H.8 Connection status icon states (user-guide §1.5)

Green dot=connected; Blue spinner=connecting/sending; Orange dot=queued pending; Red dot=connection error; Gray dot=not configured.

H.9 Detection visuals (user-guide §2.1)

Default = no page modification. Optional “Highlight torrents on page”: magnet links green underline, .torrent links blue underline, hover tooltip with name+size.

H.10 Power-user workflows (user-guide §5.4)

Nightly batch download (accumulate pages in "To Download" group → `Ctrl+Shift+G` at night); category-based org (set categories + qBT Automatic Torrent Management auto-move on completion); monitoring large batches (enable all notifications + watch).

H.11 Verification flow (installation-guide §6)

Test on known torrent site (Ubuntu downloads), verify detection (badge count, popup list name/size/source), verify send ("Send Selected" → notif), check qBT for download, queue test (disconnect→send→reconnect→item sent), settings-persist test (restart→Options retained).

I. EDGE CASES (explicitly documented)

- Magnet without `dn` → display "Unknown Torrent", name resolved by qBT from metadata (FR-003; FAQ Q16).
- Duplicate torrent (same infohash) → silently skipped, "duplicate" status; dedup merges longest displayName + union trackers + earliest timestamp (FR-006; FAQ Q17; api-ref dedup).
- Malformed tracker URLs → logged + discarded (FR-003).
- `.torrent` > 10 MB → not downloaded; `E_FILE_TOO_LARGE`(413) → "Use magnet link instead" (FR-004; api-ref §6.1).
- CORS-blocked `.torrent` download → Boba proxies (Boba mode); qBT-direct shows "CORS blocked", manual copy (FR-004; user-guide §2.3; `E_CORS` → "Use Boba proxy mode").
- Dynamically-loaded links (SPA/infinite scroll) → MutationObserver, debounced 250 ms (tech-spec §10.2) / 500 ms (impl-plan §2.3.5); "wait a few seconds" (user-guide §4.2).
- Links inside iframes → detected only when `all_frames` enabled; may be inaccessible (FR-001; user-guide §4.2).
- JS click-handler "links" (not real `<a>`) → not detected; manual copy (user-guide §4.2).
- Non-standard/encoded magnet formats → not detected; manual copy (user-guide §4.2).
- Site requires login / shows links only to logged-in users (user-guide §4.4).
- Strict-CSP pages → extension respects CSP, will NOT inject scripts on such pages (user-guide §4.2/§4.4; tech-spec threat model).

- Forgotten encryption password → no recovery; reset extension + reconfigure (user-guide §3.5; FAQ Q26; risk table).
- Browser restart without password protection → session encryption key lost → "Cannot decrypt credentials"; re-enter credentials (dev-guide §9.5).
- Storage quota exceeded → `E_STORAGE_FULL`; Firefox stricter limits, clear old queue items (api-ref §6.1; installation-guide §9.8).
- Pages with 10,000+ links → must not jank (>16 ms frame); `requestIdleCallback` for >500 links; pagination for large link sets (NFR §4.4; tech-spec §10.2; risk table).
- qBitTorrent rejects duplicate → may already be downloading/completed (user-guide §4.3).
- Private tracker torrents → work if magnet/.torrent accessible; may need passkeys in tracker URL (FAQ Q15).
- MV3 service worker termination → `chrome.alarms` keep-alive (dev-guide §6.1; risk table; impl-plan §3.2.3); Firefox suspends SW more aggressively (installation-guide §9.8).
- Self-signed certs → may need browser exception / system trust install; "HTTPS Only" can be temporarily disabled for testing (installation-guide §9.6; user-guide §4.1).
- macOS `__MACOSX` metadata folder in ZIP → load error, must delete (installation-guide §9.5).
- Mobile browsers unsupported (no full WebExtensions API / SW / tab groups) (FAQ Q4).
- Safari only "Partial" MV3, not tested (api-ref §8.2).

J. BACKEND INTEGRATION POINTS

J.1 Two operating modes

- **Boba Server mode** (recommended): extension → Boba REST `/api/v1/*` → Boba proxies/orchestrates to qBitTorrent. Adds: search aggregation, metadata enrichment, SSE progress streaming, auto-discovery, CORS proxy for `.torrent` downloads. Auth = API Key (`X-API-Key`) or JWT (`Authorization: Bearer`).
- **qBitTorrent Direct mode**: extension → qBitTorrent WebUI `/api/v2/*` directly. Auth = cookie (`SID`) via `/api/v2/auth/login`, or Basic Auth. `Referer` header mandatory. No live progress in popup.

J.2 Spec-documented endpoints the extension calls

- Send magnet/`.torrent`: Boba `POST /api/v1/torrents/download` OR qBT `POST /api/v2/torrents/add`.
- Health: Boba `GET /api/v1/health` (or `/health`); qBT `GET /api/v2/app/version`.
- Progress: Boba SSE `GET /api/v1/events/download-progress`; qBT polling `GET /api/v2/sync/maindata` + `GET /api/v2/torrents/info`.
- Search: Boba `GET /api/v1/torrents/search`.
- Categories/tags: qBT `GET/POST /api/v2/torrents/{categories,createCategory,tags,addTags,removeTags}`.

J.3 Mapping onto REAL Boba services (per task brief — flagged for conductor)

The spec assumes generic Boba ports **8443/8080**, but the real Boba repo runs: - **download-proxy on port 7186** → qBitTorrent WebUI (port 7185 internal). The qBT-direct `/api/v2/*` calls SHOULD target **7186** (the proxy), which already injects auth cookies for private trackers; WebUI creds are hardcoded `admin`/`admin`. - **merge-search service on port 7187** (FastAPI or Go/Gin) → maps to the spec's Boba search/dashboard role (`/api/v1/torrents/search`, SSE progress, dashboard at `http://localhost:7187/`). The spec's "Boba Server" REST surface is best mapped here. - **boba-jackett on port 7189** (Go) → owns Jackett credentials/indexer overrides/autoconfig; relevant to the "search aggregation" feature. - **CRITICAL FINDING — implementation-plan §4.1.4 ALREADY cites the real ports:** "Auto-discovery — Scan localhost ports **7187, 7189, 8080**." This is the strongest signal the extension's backend discovery should target Boba's real 7187/7189 (and the proxy 7186), NOT the spec's fictional 8443. - Auto-discovery probe targets in FR-011/installation-guide (`boba.local:8443`, `boba:8443`, `localhost:8443`, `:8080`) must be **re-pointed to 7186/7187/7189** for the Boba implementation.

K. BOBA-CONSTRAINT FLAGS (task-mandated callouts)

K.1 CI/CD — Boba FORBIDS all CI/CD; the reference relies heavily on GitHub Actions

The reference deliverable **assumes GitHub Actions CI/CD** in multiple places — ALL of these violate Boba's permanent Hard-Stop "NO CI/CD pipelines" rule and MUST be dropped/replaced with a manual `./ci.sh`-style script: - master-guide / README §src tree: `.github/workflows/ci.yml` (lint, test, build, E2E) and `.github/workflows/release.yml` (release to all stores). - README "Verification Checklist": "CI/CD pipelines (GitHub Actions) ". - tech-spec §12.2 "CI/CD Pipeline" mermaid: Developer Push → Lint → Unit → Build → E2E → Coverage Gate → (main) Deploy to Chrome Web Store / AMO / GitHub Release. - dev-guide §1.1: Git workflow GitHub Flow; §7.5 release process auto-submits to stores; coverage thresholds "enforced in CI". - impl-plan §8.1.3 `ci.yml` (lint/test/build) + §8.1.4 `release.yml` (publish to stores). - **Action for conductor:** strip every `.github/workflows/*.yml`; convert lint/test/build/E2E/coverage into a manual script (Boba's `./ci.sh` analogue). No auto-trigger on push/PR. Husky pre-commit hook (impl-plan §1.1.4) is ALSO a git-hook → Boba forbids git hooks → drop.

K.2 Hardcoded `localhost`/ports vs CONST-XII (no hardcoded client-facing URLs)

The spec hardwires `localhost:8080`, `boba.local:8443`, etc. Boba CONST-XII forbids hardcoded `localhost`/`127.0.0.1` for client-facing URLs — but for a browser extension the server URL is user-configured (Options), which satisfies the spirit. Discovery defaults must derive from config, not literals. Flag for review but lower severity.

K.3 Boba credentials reality

- qBitTorrent WebUI creds are **hardcoded `admin`/`admin`** in Boba — the extension's qBT-direct auth defaults can assume this for local proxy (7186) but must not commit any real secret. Spec's PBKDF2/AES-256-GCM credential encryption still applies for user-entered Boba/tracker keys.
- Never commit `.env` / tracker credentials (Boba rule) — extension build must not bundle them.

K.4 TDD / anti-bluff

Reference targets 80% unit / 60% E2E coverage with Jest+Playwright. Boba mandates TDD (RED→GREEN) + anti-bluff (assert user-observable outcomes: real HTTP to 7186/7187, real DOM text, real container state) — the reference's coverage targets are a floor; tests must drive the real proxy/merge stack, not mocks, for non-unit layers.

K.5 Tooling alignment notes

- Reference stack: WXT ≥0.17, Vite ≥5, TS ≥5, Jest ≥29, Playwright ≥1.40, ESLint ≥8 (impl-plan says ESLint 9 flat config), Prettier ≥3. Boba's existing frontend is **Angular 21** + **Vitest** — the extension is a *separate* TS/WXT codebase, so Jest-vs-Vitest is a deliberate divergence to confirm with conductor (could standardize on Vitest to match Boba).
- Runtime deps (≤20): `bencode-js ^3.0.0`, `lz-string ^1.5.0`, `webextension-polyfill ^0.10.0`. Dev deps: `@types/chrome ^0.0.260`, `@types/jest`, `ts-jest`, `jest-environment-jsdom`, `@playwright/test`.
- LICENSE: impl-plan §1.1.1 says **Apache 2.0**; master-guide §1.1.1-adjacent unclear; FAQ Q2 says "free and open-source under its license terms" — confirm with conductor.

L. DEPENDENCIES & TECH STACK (tech-spec §11; dev-guide §1.3)

Layer	Tech	Version
Language	TypeScript	≥ 5.0
Framework	WXT	≥ 0.17
Bundler	Vite (via WXT)	≥ 5.0
Unit test	Jest	≥ 29.0
E2E test	Playwright	≥ 1.40
Lint	ESLint	≥ 8.0 (impl-plan: 9 flat config)
Format	Prettier	≥ 3.0
Crypto	Web Crypto API	native (AES-256-GCM, SHA-1, PBKDF2)
Storage	Extension Storage API (MV3)	chrome.storage.local/session

Layer	Tech	Version
UI	Vanilla TS + CSS	no UI framework

Runtime deps: bencode-js ^3.0.0, lz-string ^1.5.0, webextension-polyfill ^0.10.0. **External deps (tech-spec §13.2):** Node.js 18 LTS (MIT), qBitTorrent WebUI 4.4.0 (GPL-2.0+), Boba Server 1.0.0 (proprietary), Chrome/Chromium 88, Firefox 109 (MPL-2.0). **Dev prereqs (dev-guide §1.1):** Node.js 18 LTS (rec 20 LTS), npm 9.x (rec 10.x), Git 2.40+, a supported browser. IDE: VS Code (settings + recommended extensions: Prettier, ESLint, Tailwind CSS IntelliSense, Coverage Gutters, Jest).

M. BUILD / RELEASE STEPS

M.1 Build commands (dev-guide §3)

- `npm install` (2–5 min).
- `npm run dev` → WXT dev mode + HMR → `.output/chrome-mv3-dev/`. Load unpacked per browser.
- `npm run build` → production builds: `.output/{chrome-mv3-prod, firefox-mv3-prod, opera-mv3-prod, edge-mv3-prod}/` (minified, tree-shaken, source-mapped, ZIP-ready).
- Per-browser: `npx wxt build -b chrome` / `-b firefox` / `--browser opera`.
- Browser differences via WXT browser-specific entry points (`popup/index.firefox.ts`), `import.meta.browser` conditional compilation, `webextension-polyfill`.
- `wxt.config.ts` (dev-guide §3.4): srcDir `src`, outDir `.output`, manifest perms `['activeTab','storage','contextMenus','notifications']`, `host_permissions: ['https://*/']`, action.default_popup, options_ui (open_in_tab), commands.

M.2 Test commands (dev-guide §4)

- `npm test` / `npx jest` (`--coverage`, `--watch`, single file, `--testNamePattern`). Coverage → `coverage/` (lcov-report HTML, lcov.info, coverage-summary.json).
- Coverage thresholds (CI-enforced): **Statements 80%, Branches 75%, Functions 80%, Lines 80%**. Per-file targets: api-client 90%, crypto-utils 95%, queue-manager 85%, magnet-scanner 80%, magnet-uri 95%.

- E2E: `npx playwright install chromium firefox`; `npm run test:e2e`; `--headed`, `--ui`, `--project=chromium`, single spec.

M.3 Release process (dev-guide §7.5)

Accumulate on main → release branch → bump version in manifest → update CHANGELOG.md → full test suite → Git tag → build all targets → GitHub Release → submit Chrome Web Store + AMO + Opera Addons. SemVer (Major=breaking, Minor=new features, Patch=fixes). **(All store-submission + GitHub Release auto-steps are CI/CD → see §K.1 flag.)**

M.4 Distribution channels (tech-spec §12.3)

Channel	Format	Update
Chrome Web Store	CRX	automatic
Firefox AMO	XPI	automatic
Opera Addons	CRX	automatic
GitHub Releases	ZIP	manual

M.5 Git conventions (dev-guide §7)

GitHub Flow; branch prefixes feature/bugfix/hotfix/docs/refactor/test; **Conventional Commits** (feat, fix, docs, style, refactor, test, chore, perf, security; scopes popup/options/content/background/api/queue/auth/crypto/i18n/build/deps; `!` + `BREAKING CHANGE:` footer). PR template + code-review checklist provided.

M.6 Store submission specifics (impl-plan §8.2)

Chrome Web Store (screenshots, description, privacy policy, **$5 fee**); Firefox AMO (`web-ext sign`, listing, review responses); Opera Addons (Opera build, sidebar screenshot); Yandex (Chromium-compatible package, description).

N. SECURITY ARCHITECTURE (tech-spec §8)

- **Threat model:** credential theft from disk (High → AES-256-GCM + key in session-only storage); MITM (High → HTTPS-only, optional cert pinning); XSS via malicious torrent site (Medium → CSP,

content-script isolation, no `eval()`); privilege escalation (Medium → minimal manifest, `activeTab` default); extension fingerprinting (Low → no unique identifiers to web pages).

- **Encryption flow:** user enters password/API key → random 16-byte salt → PBKDF2(100k iterations) derive key → store key in `chrome.storage.session` → AES-256-GCM encrypt → store `{iv+authTag+ciphertext, salt}` in `chrome.storage.local`. On startup: check session key; if exists use it, else re-derive from salt+password.
- **CSP:** `"extension_pages": "script-src 'self'; object-src 'none'; connect-src 'self' https:;"`.

O. ARCHITECTURE & MODULE MAP

O.1 Component layers (tech-spec §5.2)

Presentation (Popup, Options, Context Menu) → Controller (Message Router, Command Handler, Tab Manager) → Service (Parser, API Client, Queue Manager, Health) → Security (Auth Module, Crypto Module, Rate Limiter) → Storage (chrome.storage.local/session).

O.2 SW core modules (tech-spec §5.1)

Parser, Auth, API Client, Queue Manager, Health Monitor, Tab Manager, Badge Manager, Notification Manager, Crypto Module.

O.3 Directory layout (dev-guide §2.1) — canonical module list

`src/background/` (index, message-router, init); `src/content-scripts/` (index, magnet-scanner, dom-observer, torrent-parser); `src/popup/` (index.html/ts, popup.css, components: torrent-list, queue-view, status-bar, tab-groups); `src/options/` (index.html/ts, options.css, components: server-config, download-prefs, queue-settings, ui-settings); `src/modules/` (api-client, auth-manager, badge-manager, config-manager, crypto-utils, discovery-service, health-monitor, i18n, notification-manager, queue-manager, rate-limiter, state-manager, tab-manager); `src/types/` (index, messages, torrent, config, api); `src/utils/` (bencode, hash, magnet-uri, storage, time, validation); `tests/` (unit, e2e, mocks);

`_locales/`. **NOTE:** master-guide/README §src tree shows a slightly different layout (`src/parser/`, `src/scanner/`, `src/api/`, `src/shared/`, `src/content/`) — two competing layouts exist → see §K/§P ambiguities.

O.3b Performance impl (tech-spec §10)

WeakMap for DOM→torrent metadata; `URL.revokeObjectURL()` after upload; lazy module init; MutationObserver filters for link mutations only, debounced 250 ms; `requestIdleCallback` for >500 links; tree-shaking; code-splitting (Options/Popup separate chunks); shared vendor chunk; LZ-string queue compression.

O.4 Architecture decisions (dev-guide §6)

MV3 over MV2 (store requirement, ephemeral SW, lower memory; mitigate SW lifetime via `chrome.alarms` + `chrome.storage`); WXT over raw Vite (extension scaffolding, HMR, multi-browser, manifest auto-gen, file-based routing); `activeTab` over broad `host_permissions` (privacy; optional_permissions for proactive detection); AES-256-GCM (authenticated encryption, Web Crypto support, NIST standard; PBKDF2 100k iterations, 96-bit IV, 128-bit tag).

P. REFERENCE-PLAN PHASES / MILESTONES

P.1 master `plan.md` — 7 Stages (research → docs)

Stage 1 Deep Research (4 agents: Boba architecture, browser-extension APIs, torrent/magnet parsing, existing-solutions+security); Stage 2 Architecture & Technical Design (→ architecture.md); Stage 3 Implementation Plan (Phase 1–10 listing); Stage 4 Source-code dev; Stage 5 Diagrams (5 agents); Stage 6 Documentation (5 agents); Stage 7 Multi-format conversion (docx/pdf/html). Note: plan.md §“Stage 3” lists 10 phases (Phase 1 scaffold … Phase 10 docs+release) — a DIFFERENT decomposition than implementation-plan.md’s 8 phases.

P.2 `implementation-plan.md` — 8 Phases / 5-week timeline (the authoritative dev plan)

- **Phase 1 Foundation (Week 1):** 1.1 Project Setup (git init + Apache-2.0 LICENSE + README; WXT for 5 browsers; tsconfig strict + @types/chrome; ESLint 9 flat + Prettier + **Husky pre-commit**; dir

structure). 1.2 Manifest V3 (base + Chrome SW + Firefox gecko ID + Opera sidebar/minimum_opera_version + Yandex `tune://tune:` URL handling + permission model storage/alarms/notifications/activeTab/host_permissions). 1.3 Shared infra (types; logger; constants URLs/ports/regex; error classes; typed event bus).

- **Phase 2 Core Engine (Week 2):** 2.1 Magnet detect+parse (regex engine; URI decoder for dn/tr/xl; BTIH validator 40-char hex + **base32 support**; param extractor xt/dn/tr/xl/ws/xs/as/kt/mt; normalizer to `{infoHash,name,trackers}`; 50+ unit tests). 2.2 Torrent file parse (zero-dep Uint8Array bencode decoder; torrent parser class; SHA-1 infohash via Web Crypto; magnet generator; validator announce/info/piece length; tests). 2.3 DOM scanner (base class w/ EventTarget; anchor scanner `a[href^="magnet:"],a[href$=".torrent"]`; **TreeWalker text-node scanner** for inline magnets; top-20-site CSS selector DB; MutationObserver debounced 500 ms; **Shadow DOM recursive traversal**; dedup Map by infoHash; perf budget rAF yielding 16 ms/frame).
- **Phase 3 Extension Shell (Week 2–3):** 3.1 Content Script (entry; orchestrator; highlight/overlay; message handler SCAN_REQUEST; auto-scan `document_idle`; manual scan). 3.2 Background SW (entry/lifecycle; message router; chrome.alarms keep-alive; context menu “Send to Boba”/“Scan Page”/“Open Dashboard”; shortcuts; health-check loop; badge updater; notifications). 3.3 Popup (HTML/CSS theme-aware; popup.ts list/actions/state; connection status; torrent list w/ checkboxes; send + progress; batch Select All/Deselect/Invert; quick settings auto-scan/notif). 3.4 Options (HTML/CSS; options.ts save/load; Boba server config URL/port/auth/creds; **Test Connection**; **Auto-Discover** port scan; download prefs category/save-path/start-paused; security creds-encryption toggle + HTTPS enforcement).
- **Phase 4 API Integration (Week 3):** 4.1 Boba API Client (`BobaAPIClient`; auth handlers cookie/API-Key/Basic; health `/app/version`,`/api/v2/app/version`; **auto-discovery scan localhost ports 7187, 7189, 8080**; error handling backoff+classify; token-bucket rate limiter). 4.2 qBitTorrent Direct (login/SID/CSRF; add torrents; add magnets; multipart .torrent upload; monitor `/torrents/info` hash-filter; `/sync/maindata` incremental RID). 4.3 Offline Queue (`OfflineQueue`; chrome.storage.local persistence; backoff+jitter retry; chrome.alarms processor; dedup by infohash).
- **Phase 5 Tab Groups (Week 3):** 5.1 Enumeration (`chrome.tabGroups.query({})`; `chrome.tabs.query({groupId})`; URL aggregation; per-tab batch scan). 5.2 Context Menu (tab “Send

Group to Boba”; group submenu w/ per-group torrent count; scan-all+batch-send handler; progress notif).

- **Phase 6 UI/UX Polish (Week 4):** 6.1 Icons & Theming (16/32/48/128 PNG + SVG; dynamic color badges; `prefers-color-scheme`; animations/spinners). 6.2 i18n (`_locales/en/messages.json`; English strings; `chrome.i18n.getMessage()` wrapper). 6.3 Accessibility (ARIA labels; keyboard nav; focus mgmt/trapping; screen-reader live regions).
- **Phase 7 Testing (Week 4–5):** 7.1 Unit (Jest+ts-jest+jsdom; chrome.* mocks storage/tabs/runtime/alarms; magnet 50+; torrent fixtures; DOM scanner jsdom; API client mock fetch; queue persistence/retry/dedup; **80%+ line coverage**). 7.2 Integration (against real Boba docker; cookie+API-key auth; **end-to-end Detect→Parse→Send→Verify download**). 7.3 E2E (Playwright config w/ extension loading; extension fixture Chromium/Chrome; popup tests; content-script detection tests; options configure+test-connection).
- **Phase 8 Build & Distribution (Week 5):** 8.1 Build pipeline (WXT prod build; cross-browser packages; **`.github/workflows/ci.yml`**; **`release.yml`** publish to stores). 8.2 Store submission (Chrome WS $5 fee; AMO web-ext sign; Opera Addons; Yandex).

P.3 Gantt (impl-plan)

Wk1: P1+P2start; Wk2: P2+P3start; Wk3: P3+P4+P5; Wk4: P6+P7start; Wk5: P7+P8.

P.4 Risk assessment (impl-plan + tech-spec §14)

qBitTorrent API changes (High/Med → version detection, adapter pattern); CORS on torrent sites (High/Med → SW fetch bypasses CORS / proxy via Boba); MV3 SW termination (Med → chrome.alarms keep-alive); store review rejection (Low/Med → minimal permissions, guideline compliance); Yandex API differences (Low → Chromium-compatible code); Chrome MV3 policy changes (Med/High → MV2 fallback branch); user forgets encryption password (Med → optional hint, reset/reconfigure); third-party dep vulnerability (Med/High → Dependabot, lockfile pinning, SCA); large-page perf (Med → requestIdleCallback, pagination).

Q. ACCEPTANCE CRITERIA / VERIFICATION (consolidated)

- FR-001: dynamic magnet links detected within **500 ms**; all http/https origins; iframe when all_frames.
- FR-003: `xt` valid 40-char hex required; `dn` fallback “Unknown Torrent”; malformed `tr` discarded.
- FR-005: computed SHA-1 infohash matches `xt`.
- NFR targets (§C) are pass/fail acceptance gates with measurement methods.
- Installation verification checklist (installation-guide §6.4): extension installed (icon), server connected (green badge), detection works (badge count >0), send works (success notif), queue works (disconnect→send→reconnect→sent), settings persist (restart→retained).
- Connection-test success output (installation-guide §4.3): Server / Mode / Status=Connected / Version / Latency / Auth=OK / API=Compatible.
- README “Verification Checklist” (24 items) — research, plan, source, SQL, unit (320+, 80%+), E2E (80+), CI/CD(GHA —), spec (25 FR/15 NFR), API ref, user guide (30 FAQs), dev guide, install guide, all diagram types, multi-format docs.

R. VERSION COMPATIBILITY MATRICES (api-ref §8; installation-guide §7.3)

- **qBitTorrent:** 4.4.x (WebAPI 2.8.x, Compatible/legacy); 4.5.x (2.8.x, recommended min); 4.6.x (2.9.x, primary target); 5.0.x (2.10.x, compatible); 5.1.x (2.11.x, testing); <4.4.0 unsupported.
- **Browsers:** Chrome 88 min/120+ rec (full MV3, tested); Firefox 109/121+ (full w/ polyfill, tested); Opera 74/106+ (full, tested); Yandex 21/24+ (full, tested); Edge 88/120+ (full, tested); Safari 16/17+ (partial, NOT tested).
- **Boba Server:** 1.0.x↔ext 1.0.x compatible; 1.1.x backward-compatible; 1.2.x testing; 2.0.x↔ext 1.1.x+ planned.
- **Feature availability:** SSE progress = Boba 1.1+ only; search aggregation = Boba 1.2+ only; auto-discovery = Boba 1.0+ only; magnet send / .torrent upload / categories / tags / sequential / first-last-piece = qBT 4.4+ (tags 4.3.2+) and all Boba.

S. RATE LIMITING DETAIL (api-ref §7)

Token-bucket `TokenBucketRateLimiter(capacity, refillRate, initialTokens)`. Default buckets: per-second cap 10 refill 10/s; per-minute cap 60 refill 1/s; queue-processing cap 1 refill 2/s. Respects `X-RateLimit-Limit/Remaining/Reset` + `Retry-After`. Queue config: maxConcurrent 1, interCallDelayMs 500, batchSize 10, batchIntervalMs 5000.

T. OPEN QUESTIONS / AMBIGUITIES (for conductor)

1. **Real Boba port mapping (HIGH).** Spec uses fictional 8443/8080; implementation-plan §4.1.4 already names **7187, 7189, 8080**. Confirm: qBT-direct → proxy **7186**? Boba REST/search/SSE → merge service **7187**? search aggregation → **boba-jackett 7189**? All FR-011 / installation-guide discovery defaults must be re-pointed. The spec's separate Boba `/api/v1/*` REST surface (auth/token, search, download, SSE) may not exist on 7187 as specified — needs reconciliation with the actual FastAPI/Go routes.
2. **CI/CD removal (HIGH — Boba Hard-Stop).** Reference is GitHub-Actions-centric (ci.yml, release.yml, Husky pre-commit, coverage "enforced in CI", auto store-submission). ALL must be converted to a manual `./ci.sh`-style script + manual release; no `.github/workflows/`, no git hooks. Confirm the manual-CI shape.
3. **Jest vs Vitest (MED).** Reference mandates Jest+Playwright; Boba's existing `frontend/` is Angular 21 + **Vitest**. Standardize the extension on Vitest to match Boba, or keep Jest? Playwright E2E likely stays.
4. **Two divergent source layouts (MED).** master-guide/README src tree (`src/parser`, `src/scanner`, `src/api`, `src/shared`, `src/content`) vs dev-guide §2.1 (`src/modules`, `src/utils`, `src/content-scripts`). Pick one canonical layout.
5. **Two divergent phase plans (MED).** plan.md lists 10 phases; implementation-plan.md lists 8. Use the 8-phase impl-plan as authoritative?
6. **Data-model vocabulary drift (MED).** SQL schema `source_type` adds `'torrent-url'`; `status` uses `'completed'` (TS uses `'dead-letter'`); `auth_method` adds `'none'`, omits `'custom-header'`. TS `QueueStatus` and SQL `status` disagree. Reconcile the canonical enum sets.

7. **Magnet param set drift (LOW).** FR-003 lists xt/dn/tr/xl/ws/x.pe; impl-plan §2.1.4 lists xt/dn/tr/xl/ws/**xs**/**as**/**kt**/**mt** (no x.pe). Which param superset is implemented?
8. **Keyboard-shortcut count (LOW).** FR-017 defines 3 commands; user-guide §2.7 + impl-plan add a 4th "Scan page (no send)" `Ctrl+Shift+S`. Confirm 3 or 4 commands; `open-popup` `Ctrl+Shift+L` may collide with browser defaults.
9. **`host_permissions` scope (LOW–MED).** wxt.config uses `host_permissions: ['https://*/']` while NFR-012 + dev-guide §6.3 push `activeTab`-primary/minimal. Reconcile (broad https → store-review risk + privacy prompt).
10. **LICENSE (LOW).** impl-plan says Apache 2.0; FAQ says "open-source under its license terms". Confirm license + Boba submodule conventions.
11. **Naming (LOW).** Product is "BobaLink" throughout the reference; confirm the Boba repo's preferred extension name + icon ("tea bubble").
12. **Search/SSE feature scope (MED).** Reference's richest features (search aggregation, SSE live progress, auto-discovery) are gated to Boba-server mode. Does the Boba implementation expose `/api/v1/torrents/search` + SSE on 7187, or must the extension call the existing merge-service routes (different shape)? Determines how much of FR-011/§E.2 is buildable as-specified.
13. **Boba `/api/v1/health` vs real `/health`.** FR-024 + discovery probe `/health`; spec also uses `/api/v1/health`. Confirm the real Boba health endpoint path/port.

End of extraction 01-guides-and-plan.md